

Project — Creative assistant

LESSON 09 · MODULE 2 CAPSTONE — MULTIMODAL

What you'll learn

- How to compose three specialized agents into a "plan, then fan out" pipeline
- Why a typed `creative_brief` is the right interface between the planner and the producers
- When to use the flagship model vs a cheap orchestrator inside the same system
- How to write image prompts that don't get sabotaged by the renderer adding logos or UI
- The architectural template you'll keep reusing: small typed specialized agents beat one big one

Key concepts

The pipeline. User gives a one-line product description. The **brief agent** (structured output, video 05) returns a typed `CreativeBrief` — name, headline, key benefits, visual concept. The **image agent** (image generation, video 07) reads the visual concept and renders a hero PNG. The **doc agent** (file generation, video 08) reads the name, headline, and benefits and renders a marketing PDF. Three agents, one orchestrator, two artifacts.

Plan, then fan out. The planning step makes the brief. The fan-out runs the image and the doc against the *same* plan. Each agent has one job. This is the architectural template — same shape as the classify-then-answer pattern in video 06.

Use the flagship model where the thinking happens. Brief agent gets the heavy lifter (GPT-5.2) because it's doing real creative reasoning. Image agent gets GPT-5 Mini because its only job is to pass the visual concept to the renderer. Doc agent gets the flagship again because writing copy is its actual job. Match the orchestrator to the work.

Field descriptions steer the model. In the brief's `visual_concept`, the description specifies "no screenshots, no UI, no laptop screens" — because image models will sneak a logo or a label into any software-product render if given room. The image agent then reads this description, writes its prompt to match, and the wrapper repeats the rules at prompt level for belt-and-suspenders.

Types travel between agents, English crosses the model boundary. The brief is typed for the planning step. The doc agent flattens the brief back into natural language before handing it to the model. The structured brief is for *you*; the model reads English.

TAKEAWAY

The pipeline.

TAKEAWAY

Plan, then fan out.

TAKEAWAY

Use the flagship model where the thinking happens.

TAKEAWAY

Field descriptions steer the model.

TAKEAWAY

Types travel between agents, English crosses the model boundary.

Code shown in this lesson

PYTHON

```
from pydantic import BaseModel, Field

class CreativeBrief(BaseModel):
    product_name: str
    headline: str
    key_benefits: list[str] = Field(description="3 short benefit statements.")
    visual_concept: str = Field(description=(
        "A photo brief – subject, lighting, composition, style. "
        "No screenshots, no UI, no laptop screens. "
        "If software, describe a metaphor or scene that evokes the product."
    ))

brief_agent = studio.create_agent(
    name="brief-agent",
    provider="gpt-5.2",                      # flagship – real creative thinking
    role="senior creative director",
    goal="distill a product description into a shippable brief",
    instructions="Tight, vivid, no fluff.",
    response_model=CreativeBrief,
)
```

PYTHON

```
from lyzr.image_models import GeminiImage

image_agent = studio.create_agent(
    name="image-agent",
    provider="gpt-5-mini",                  # cheap orchestrator
    image_model=GeminiImage.gemini_pro,    # Nano Banana Pro
    role="image prompt renderer",
    goal="render the brief's visual concept",
    instructions="Pass the visual concept verbatim. No logos. No text in image.",
)

doc_agent = studio.create_agent(
    name="doc-agent",
    provider="gpt-5.2",                    # flagship – copy is the job
    role="marketing copy writer",
    goal="produce a one-page marketing PDF",
    instructions="Match the headline tone. Use the benefits as a bulleted block.",
    file_output=True,
)
```

PYTHON

```
def creative_assistant(product_description: str):
    # Plan
    brief = brief_agent.run(product_description)
    print(f"BRIEF: {brief.product_name} – {brief.headline}")

    # Fan out – image
    image_prompt = (
        f"{brief.visual_concept}. "
        "High quality, no text, no logos, no UI."
    )
    image_response = image_agent.run(image_prompt)
    image_path = image_response.files[0].download(safe_file_name(brief.product_name + ".png"))

    # Fan out – doc
    doc_prompt = f"""
    Write a one-page marketing PDF for:
    Product: {brief.product_name}
    Headline: {brief.headline}
    Benefits: {'', '.join(brief.key_benefits)}
    Format as a PDF.
    """
    doc_response = doc_agent.run(doc_prompt)
    doc_path = doc_response.files[0].download(safe_file_name(brief.product_name + ".pdf"))

    return image_path, doc_path
```

TRY THIS

- 1 Add a fourth agent — `social_post_agent` — that takes the brief and writes a LinkedIn caption (text-only, no `file_output`, no `image_model`). Wire it into the orchestrator after the doc agent. Now one product description fans out to three artifacts.
- 2 Cache the brief from one run, then call the image agent three times in a row with different image models (DALL-E 3, Gemini Pro image, GPT-Image-1). One plan, three renders. Compare the aesthetic register — which model honored the `visual_concept` most literally?